Project 2

Jason Soto

```
here::i_am("Projects/project2.qmd")
library(ggplot2)
library(dplyr)
library(tidymodels)
library(broom)
library(tidyverse)
library(rsample)
library(tidyclust)
library(dendextend)
library(factoextra)
library(kknn)
library(forcats)
library(MASS)
library(nnet)
```

Section 1: Data Description

Write 1-2 paragraphs describing the data, without doing anything other than importing it or minor data cleaning (e.g., changing variable names or variable types). Important questions to answer are listed in the "Rubric Item" table below.

Part a

What is an observation in this dataset? How many observations are in this dataset? How many variables?

An observation in this dataset presents a Pokemon as featured in their debut game, rather than any later appearances or alternate form (Deoxys). The various game stats data included are their base stats (HP, attack, defense, special attack, special defense, and speed) as featured in their debut game, along with a total base stat value, height, weight, Pokedex ID, and their generation id . There are 649 observations with 15 different variables.

Part b

Explain briefly who collected the data and how/where/why the data was collected.

This CRAN dataset was collected by William Amorim and published in 2023. Stats data was collected from Bulbapedia, a reliable and community-driven online encyclopedia dedicated to the Pokemon franchise, and from a GitHub that hosts data for Pokemon as well. A separate GitHub was used to link to Pokemon images, we removed that column and other unnecessary columns.

Part c

Import the data into R.

```
library(pokemon)

pokemon <- pokemon |>
  dplyr::select(
    id, pokemon, height, weight, base_experience, type_1, type_2,
    hp, attack, defense, special_attack, special_defense, speed,
    generation_id
  ) |>
  dplyr::filter(generation_id < 6) |>
  mutate(total_stats = hp + attack + defense + special_attack + special_defense + speed)
```

This data set contains relevant stats on various Pokemon. Indexing issues are caused by Mega Pokemon and by alternate versions of Pokemon, so I subsetted the dataset. Another reason for this was due to my lack of interest in the Pokemon past the 5th generation. So, Pokemon removed were any beyond the 5th generation, a few variables were removed too, such as 2 columns with urls to images, 3 columns that contained hex codes to their main colors, species id variable, and the 2 egg group variables. This is due to the fact that they will be useless here. The species id variable was removed since it was the same has the id variable, and egg groups are just irrelevant to my narrative. I also added a total stats variable that sums up all 6 of their base stats for a singular variable.

Part d

Split the data into a training and test set

```r
set.seed(67)

pokemon_split <- initial_split(pokemon, prop = .8)

pokemon_train <- training(pokemon_split)
pokemon_test <- testing(pokemon_split)
```

Part e

Justify how the training/test split was done, including the size of each set.

The split proportion is the standard 80% and 20% random split into training and testing sets. With 649 observations, this leads to 519 Pokemon in the training set and 130 in the test set.

Section 2: Data Wrangling and Exploratory Data Analysis

Think about the kind(s) of question(s) you would like to ask on the portfolio project. Perform data wrangling and exploratory data analysis on the training set with the goal of gaining insight into the question(s).

Note to self: I have three ideas I want to test, 1. Do types tend to have patterns in their base stats? (ex. maybe rock have better defense? Stuff like that) 2. Which types tend to have a superior average total of stats in each generation? 3. Can we predict a pokemon's type based off stats? 4. Cluster similar Pokemon I guess?

(clustering or classification based on stats)

Part a

Produce univariate summaries and graphs of at least two variables (features and/or outcomes)

```r
summary(pokemon_train$total_stats)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  190.0   320.0   425.0   416.8   497.5   680.0 
```

```r
summary(pokemon_train$hp)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   1.00   50.00   65.00   69.05   80.00  255.00
```

```
summary(pokemon_train$attack)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   5.00   51.50   73.00   74.31   93.50  165.00
```

```
summary(pokemon_train$defense)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   5.00   50.00   65.00   70.48   85.00  230.00
```

```
summary(pokemon_train$special_attack)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  10.00   45.00   65.00   68.32   90.00  154.00
```

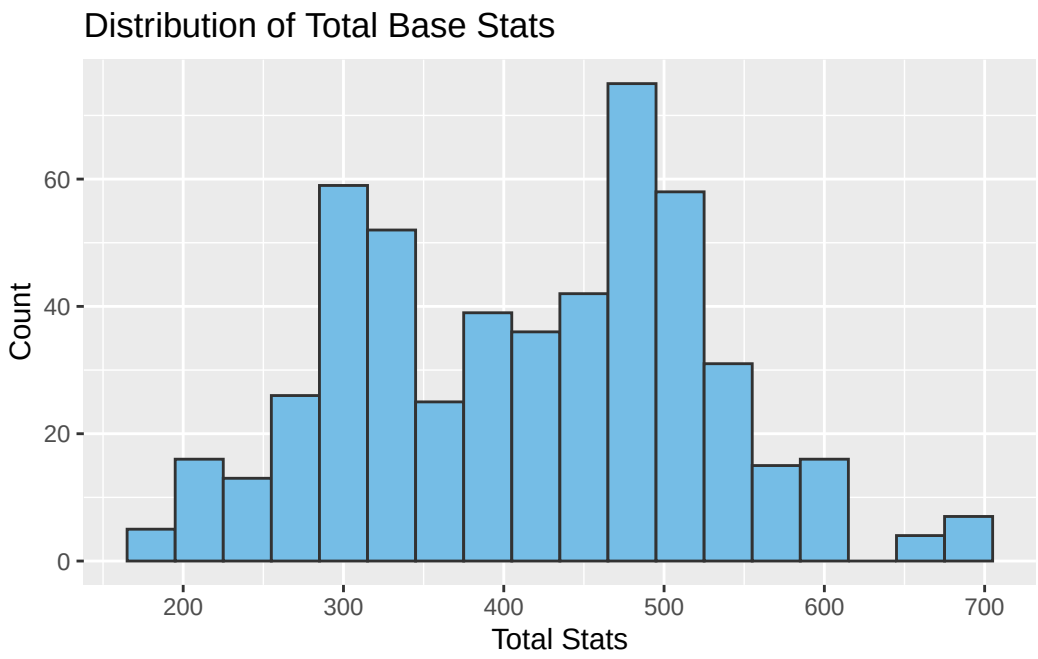
```
summary(pokemon_train$special_defense)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  20.00   50.00   65.00   69.04   85.00  230.00
```

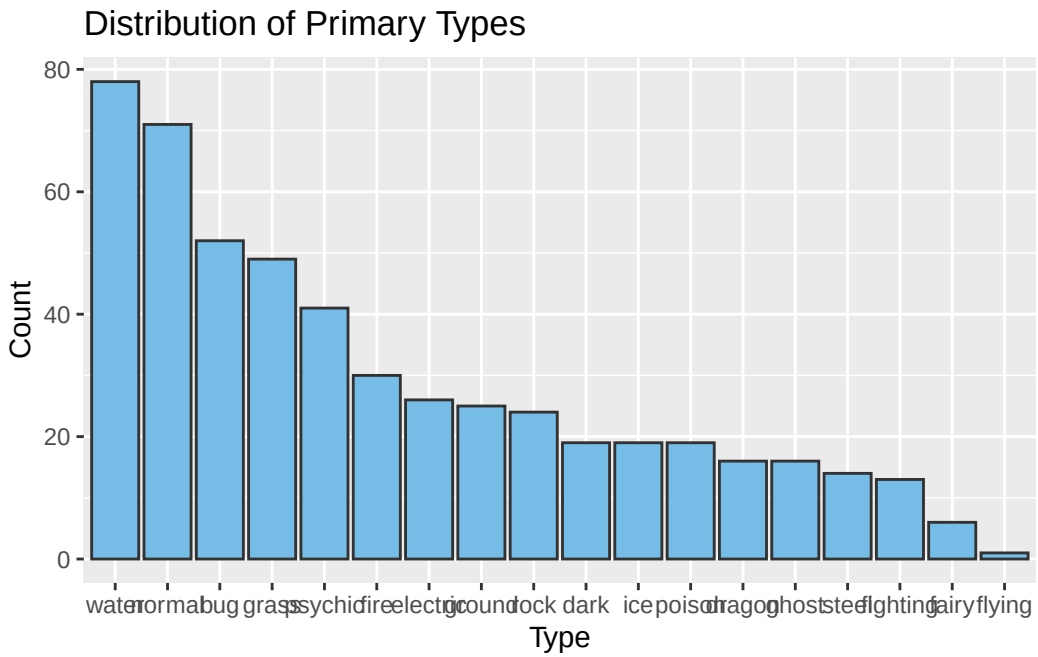
```
summary(pokemon_train$speed)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   5.00   45.00   65.00   65.56   85.00  160.00
```

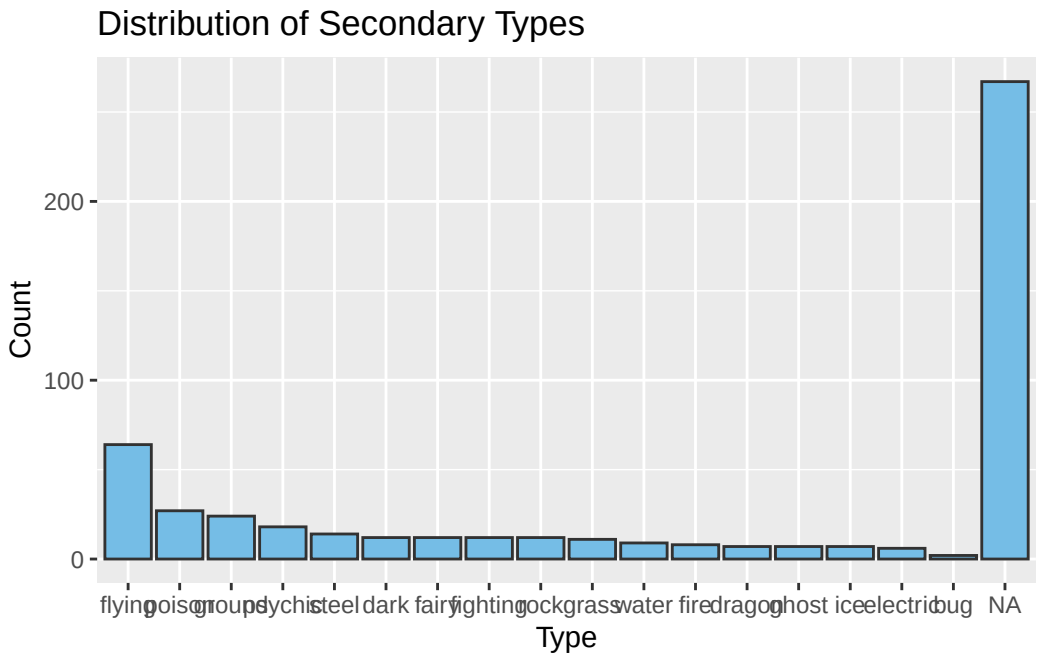
```
ggplot(pokemon_train,
       aes(x = total_stats)) +
  geom_histogram(binwidth = 30,
                 fill = "#75bde6",
                 color = "#323232") +
  labs(title = "Distribution of Total Base Stats",
       x = "Total Stats",
       y = "Count")
```




```
ggplot(pokemon_train,
       aes(x = fct_infreq(type_1))) +
  geom_bar(fill = "#75bde6",
           color = "#323232") +
  labs(title = "Distribution of Primary Types",
       x = "Type",
       y = "Count")
```




```
ggplot(pokemon_train,
       aes(x = fct_infreq(type_2))) +
  geom_bar(fill = "#75bde6",
           color = "#323232") +
  labs(title = "Distribution of Secondary Types",
       x = "Type",
       y = "Count")
```



The few outliers on the right in the Distribution of Total Base Stats may be legendaries, pseudo-legendaries, and mythicals.

Part b

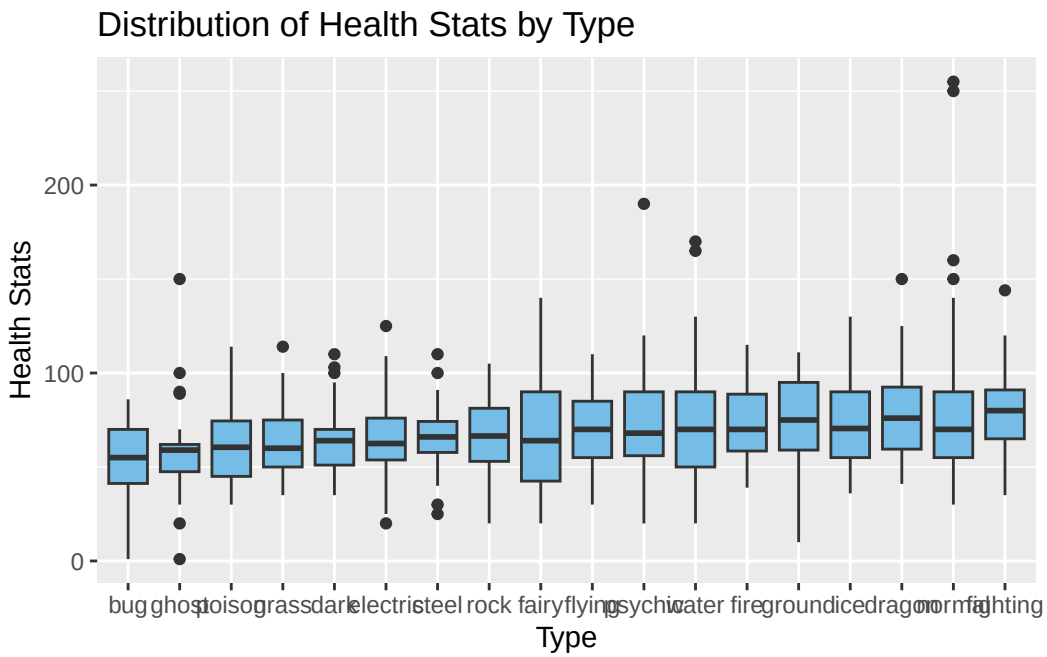
Produce at least two graphs showing bivariate or multivariate trends (or lack thereof)

```
pokemon_types <- pokemon_train |>
  pivot_longer(cols = c(type_1, type_2),
               names_to = "type_slot",
               values_to = "type") |>
  filter(!is.na(type))
```

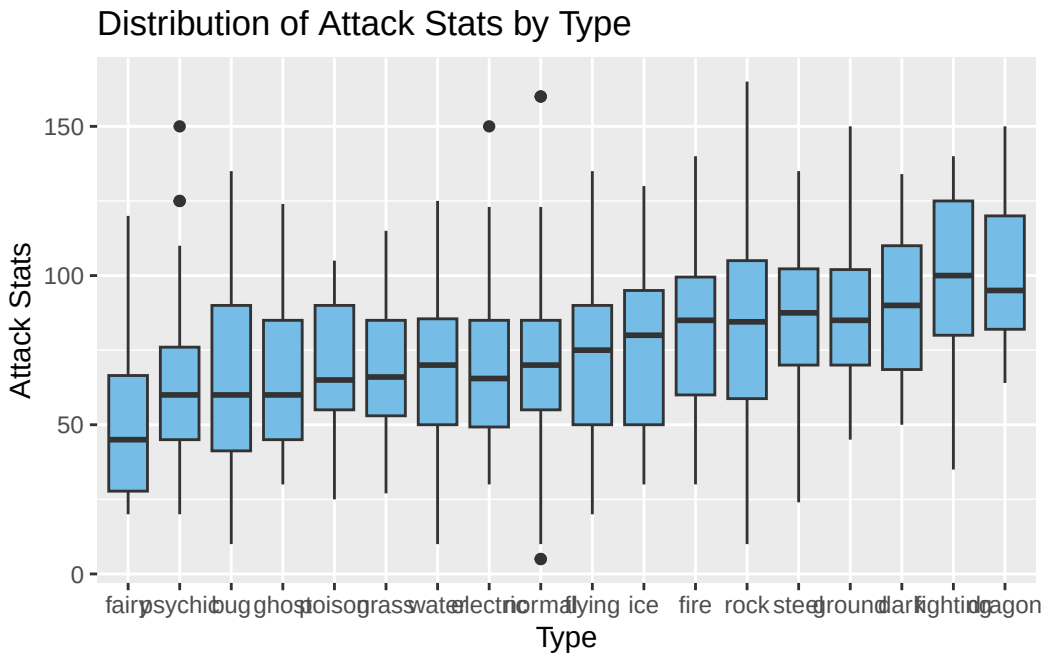
What this code does is double count each pokemon if they have two typings, that way their secondary type is counted towards the graphs in a better way. This is because for example, we only have one primary flying, with many secondary flying, and I want to display cleaner pattering without a dozen graphs.

```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, hp, .fun = mean),
           y = hp)) +
```

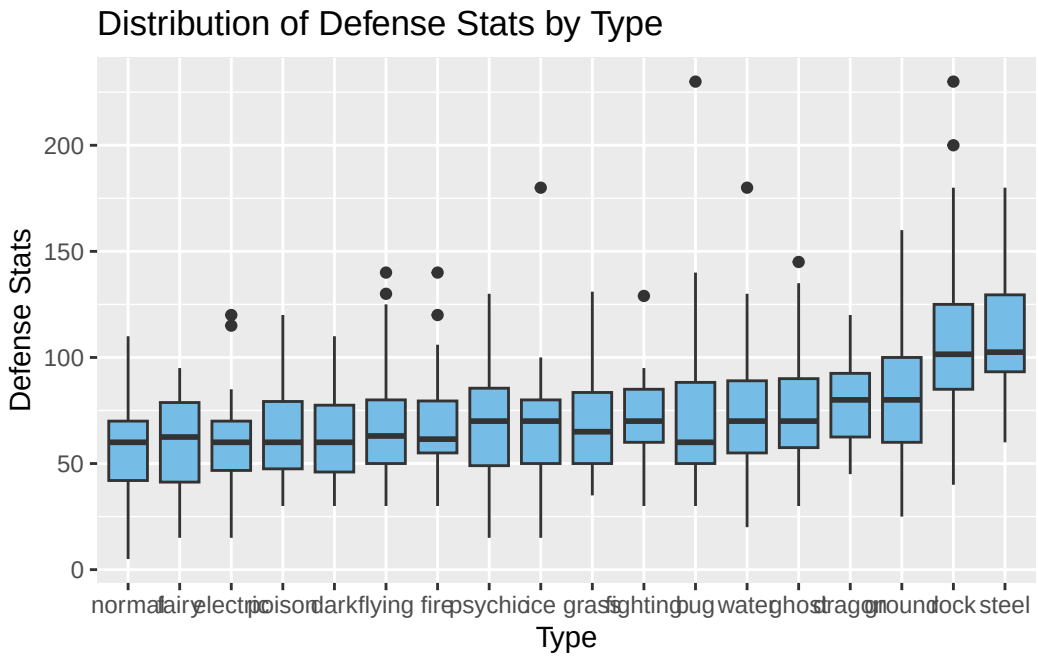
```
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Health Stats by Type",
       x = "Type",
       y = "Health Stats")
```



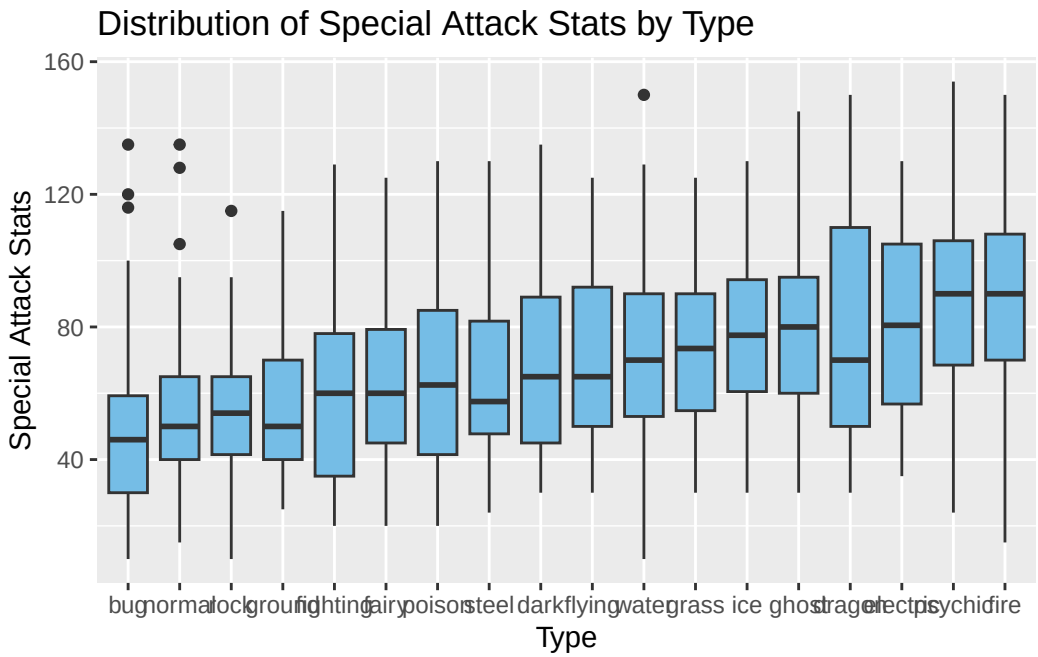

```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, attack, .fun = mean),
           y = attack)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Attack Stats by Type",
       x = "Type",
       y = "Attack Stats")
```



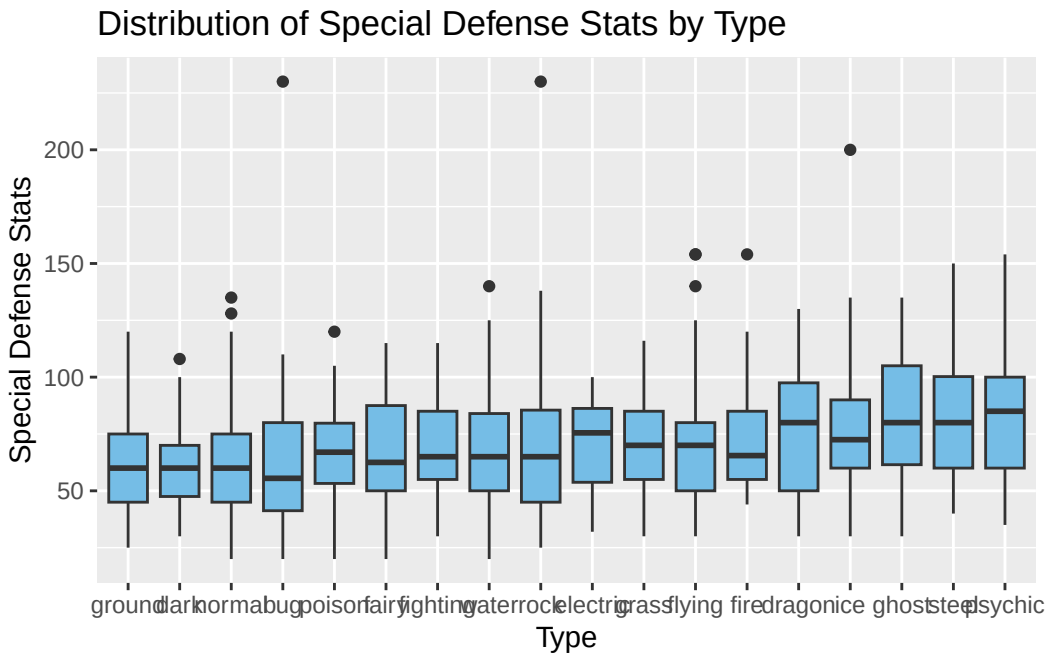

```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, defense, .fun = mean),
           y = defense)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Defense Stats by Type",
       x = "Type",
       y = "Defense Stats")
```



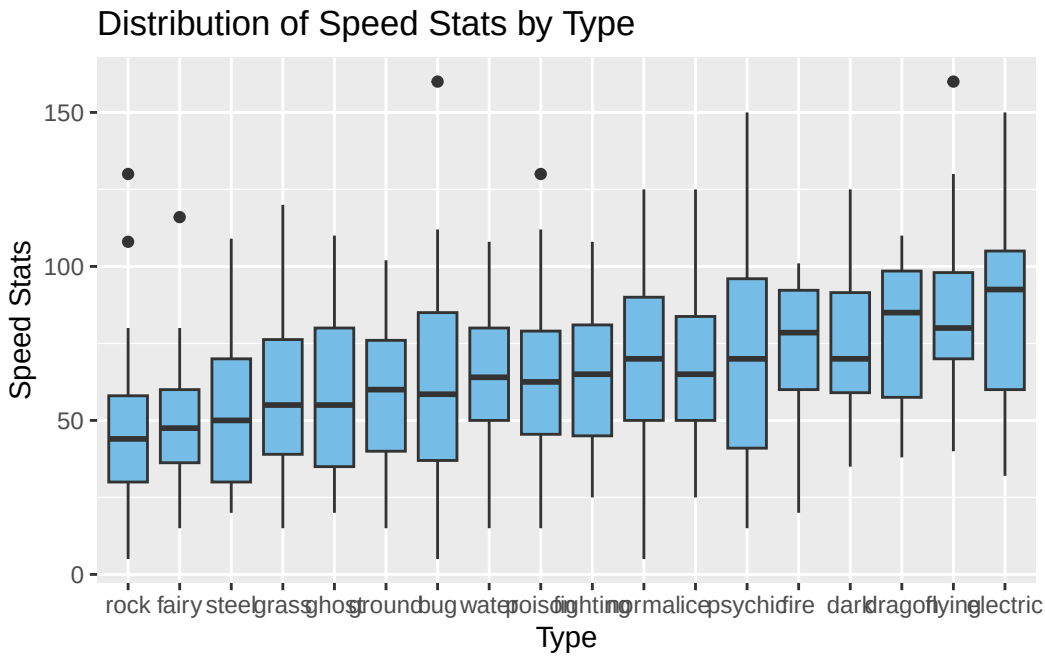

```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, special_attack, .fun = mean),
           y = special_attack)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Special Attack Stats by Type",
       x = "Type",
       y = "Special Attack Stats")
```




```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, special_defense, .fun = mean),
           y = special_defense)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Special Defense Stats by Type",
       x = "Type",
       y = "Special Defense Stats")
```




```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, speed, .fun = mean),
           y = speed)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Speed Stats by Type",
       x = "Type",
       y = "Speed Stats")
```



Part c

Write a narrative describing what was explored, why, and what was found (in context). Interleave the code and graphs with the narrative in the appropriate places. Include follow-up questions and answers as appropriate to the narrative.

The purpose of this exploration was to see if certain Pokemon types showed patterns or superiority towards certain base stats. One idea that a player might have is that dragon types are one of the best types, we will see that pattern.

First, on average Pokemon with better health stats tend to be fighting, normal, and dragon. As for attack stats, dragon and fighting take it as they are more physical Pokemon, while psychic is near the end due to its more "special" specialty, with it being psychic and all. Next, it is very clear why steel and rock immensely over shadow the rest in defense, with ground and dragon following behind. As for special attack we have fire, psychic, electric, and dragon. Special defense on average is very uniform, but we do have psychic steel and ghost in the lead. Finally, we have electric, flying, and dragon in the lead for speed which makes much sense. So if you were to go up against any Pokemon of those three types, they're more likely to go first.

Overall, this analysis shows that Pokemon types may very be associated or correlated with certain statistical patterns to some extent rather than being randomly distributed across types. Statistics are even distributed in ways one could guess.

Part d

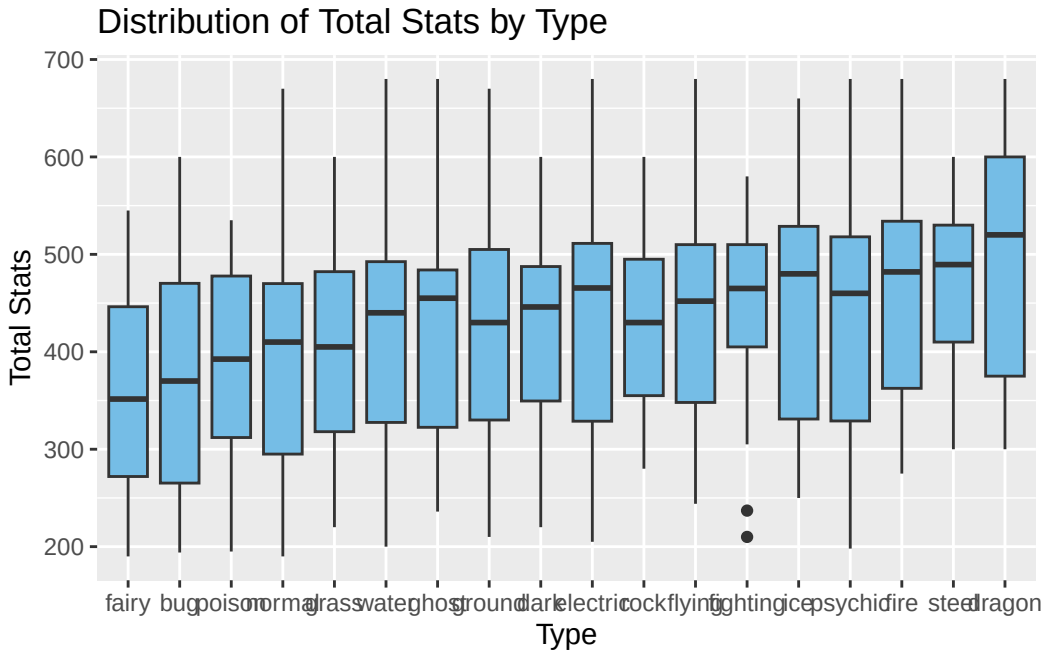
Explore in-depth one or more follow-up questions as appropriate to the original narrative.

A possible follow-up question is whether certain Pokemon types are stronger overall based off their total base statistics, rather than individual attributes. Another idea is we can see which type is the strongest overall for each individual generation.

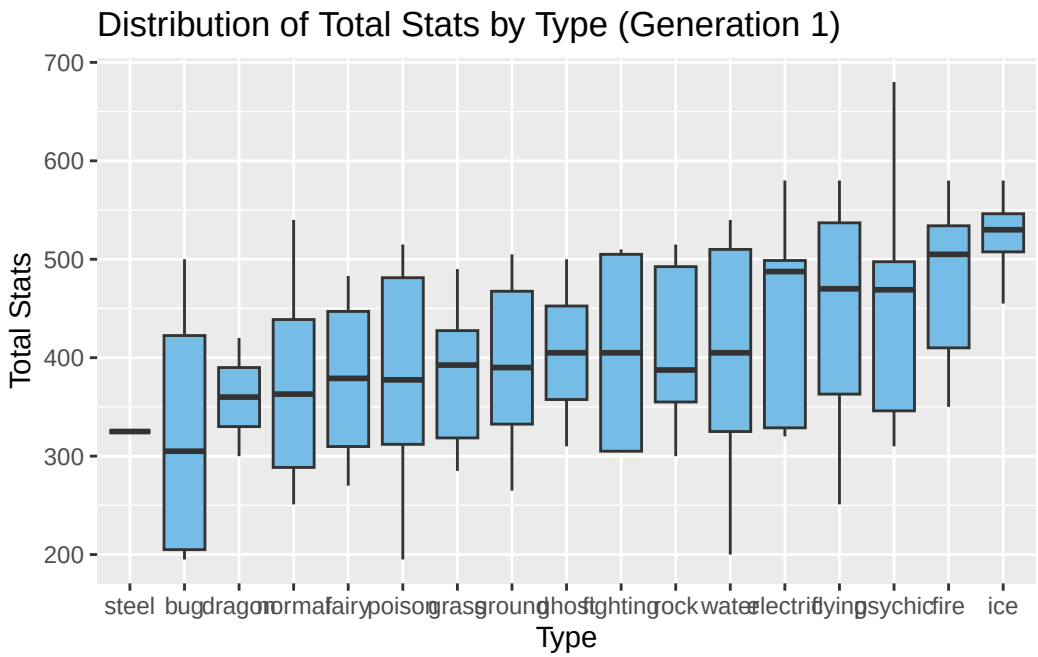
Part e

Write a narrative describing what you followed up on, why, and what you found (in context). Interleave the code and graphs with the narrative in the appropriate places.

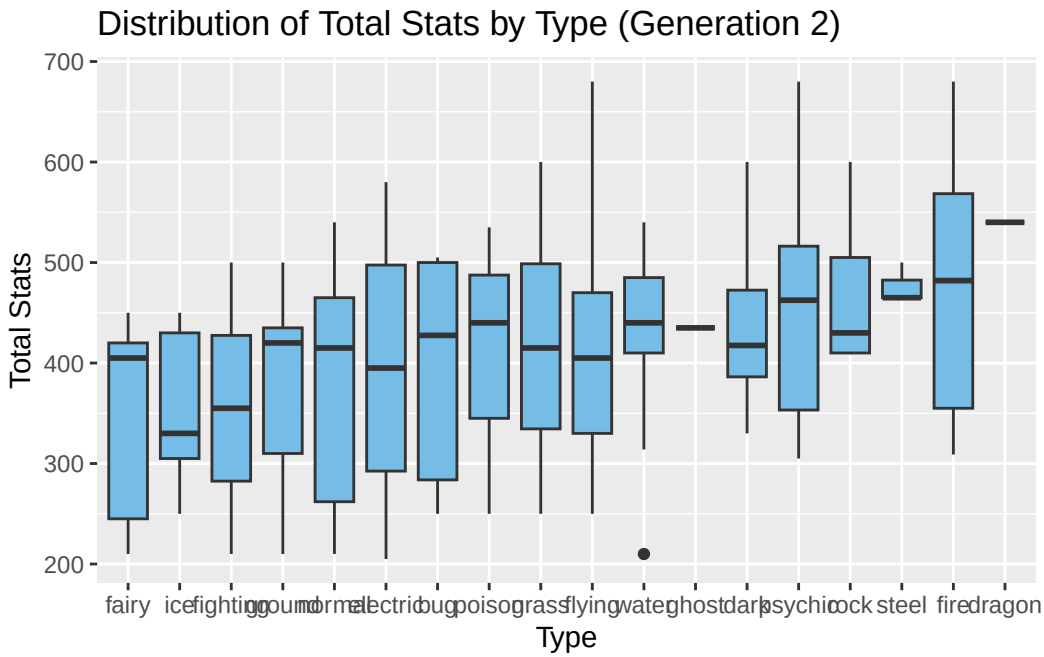
```
ggplot(pokemon_types,
       aes(x = fct_reorder(type, total_stats, .fun = mean),
           y = total_stats)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Total Stats by Type",
       x = "Type",
       y = "Total Stats")
```



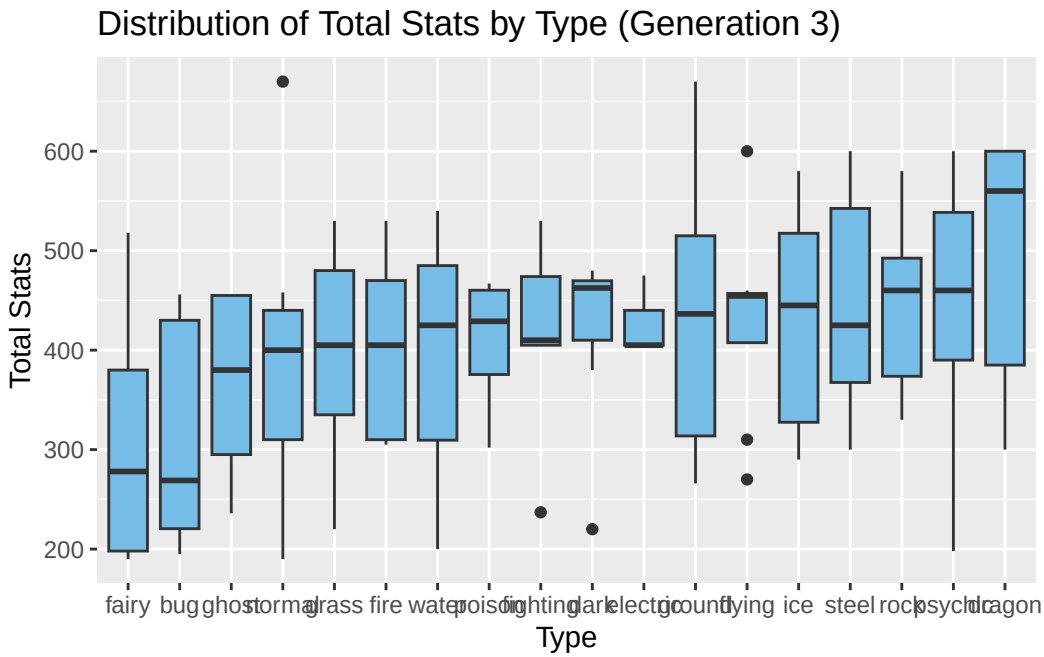
```
ggplot(pokemon_types |> filter(generation_id == 1),
       aes(x = fct_reorder(type, total_stats, .fun = mean),
           y = total_stats)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Total Stats by Type (Generation 1)",
       x = "Type",
       y = "Total Stats")
```



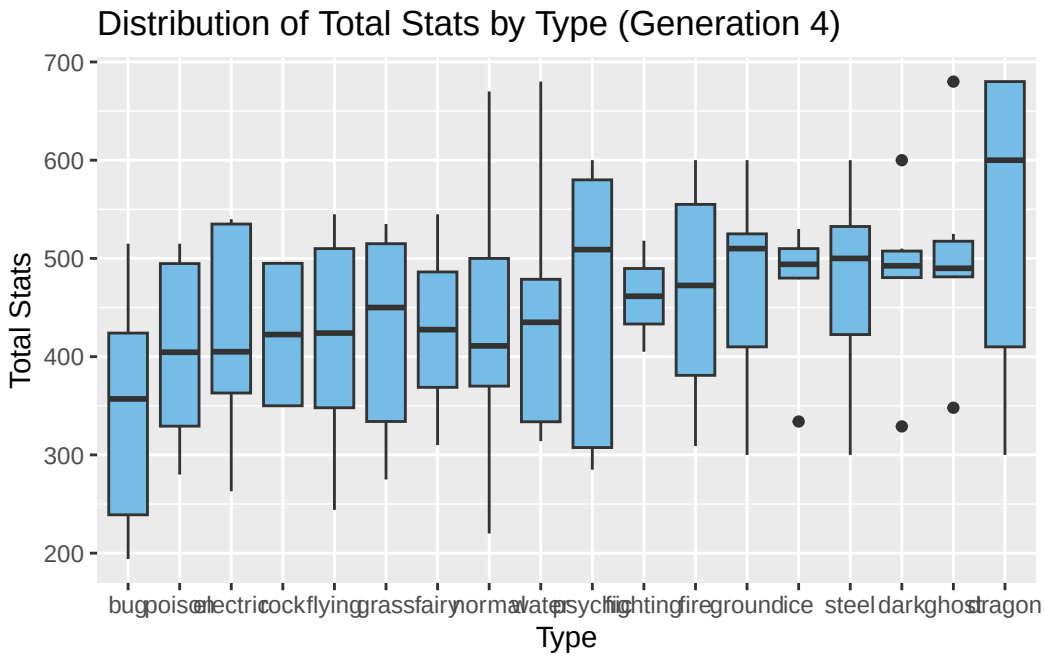

```
ggplot(pokemon_types |> filter(generation_id == 2),
       aes(x = fct_reorder(type, total_stats, .fun = mean),
           y = total_stats)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Total Stats by Type (Generation 2)",
       x = "Type",
       y = "Total Stats")
```



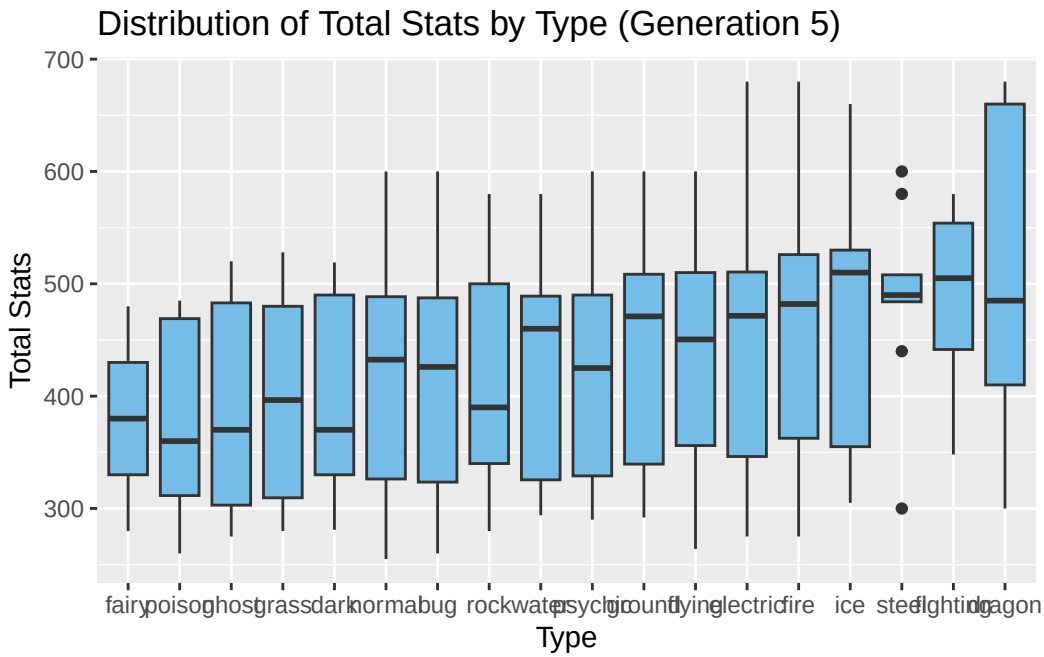

```
ggplot(pokemon_types |> filter(generation_id == 3),
       aes(x = fct_reorder(type, total_stats, .fun = mean),
           y = total_stats)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Total Stats by Type (Generation 3)",
       x = "Type",
       y = "Total Stats")
```




```
ggplot(pokemon_types |> filter(generation_id == 4),
       aes(x = fct_reorder(type, total_stats, .fun = mean),
           y = total_stats)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Total Stats by Type (Generation 4)",
       x = "Type",
       y = "Total Stats")
```




```
ggplot(pokemon_types |> filter(generation_id == 5),
       aes(x = fct_reorder(type, total_stats, .fun = mean),
           y = total_stats)) +
  geom_boxplot(fill = "#75bde6") +
  labs(title = "Distribution of Total Stats by Type (Generation 5)",
       x = "Type",
       y = "Total Stats")
```




```
pokemon_types |> dplyr::count(type, sort = TRUE)
```

```
# A tibble: 18 x 2
   type         n
   <chr>    <int>
 1 water       87
 2 normal      71
 3 flying      65
 4 grass       60
 5 psychic     59
 6 bug         54
 7 ground      49
 8 poison      46
 9 fire        38
10 rock        36
11 electric    32
12 dark        31
13 steel       28
14 ice         26
15 fighting    25
16 dragon      23
17 ghost       23
18 fairy       18
```

Overall, dragon types are the most superior Pokemon, followed by steel and fire. However, dragon Pokemon superiority may be due to the fact that there is a small number of them due to their pseudo-legendary status, and because of their status, they are intentionally designed to be powerful Pokemon rather than the type itself causing higher stats. Next, when we check by each generation, based off of just the Total Stats, dragon remains the best type except for in the 1st generation, where the best type is ice. Finally, as I mentioned, the count for each type may cause possible biases in the box plots, so I included that code just for one to take note of.

In conclusion, we should not determine these data analysis as absolute truth and superiority, but as said in the beginning, we should probably just interpret these results as patterns as these are just averages.

Section 3: Unsupervised Learning

Select one of the following techniques: Principal Component Analysis, k-means clustering, or model-based clustering. You may additionally choose hierarchical clustering if your training set is small enough that you can get a reasonable-looking dendrogram.

Part a

Perform the technique on the training set. Select the number of PCs or number of clusters. Justify your selection.

```
pokemon_clust <- pokemon_train |>
  dplyr::select(hp, attack, defense, special_attack, special_defense, speed)

x.matrix <- scale(as.matrix(pokemon_clust))

set.seed(67)

km.pokemon <- kmeans(x.matrix, centers = 4, nstart = 50)

augmented_pokemon <- augment(km.pokemon, pokemon_train)

table(augmented_pokemon$.cluster)
```

```
  1   2   3   4
118  79 105 217
```

```
km.pokemon$centers
```

```
          hp     attack    defense special_attack special_defense      speed
1  0.1267645  0.3852633 -0.0552137      0.9971806       0.3614969  1.1822494
2  0.8883215 -0.1756910  0.7093584      0.5902211       1.2849417 -0.4431938
3  0.5869001  1.1423760  0.7812450     -0.1668296       0.0492475  0.1078808
4 -0.6763139 -0.6982994 -0.6062434     -0.6763948      -0.6881936 -0.5337355
```

```
augmented_pokemon |>
  dplyr::select(pokemon, type_1, type_2, .cluster, hp, attack, defense,
                special_attack, special_defense, speed) |>
  arrange(.cluster)
```

```
# A tibble: 519 x 10
   pokemon            type_1 type_2 .cluster    hp attack defense special_attack
   <chr>              <chr>  <chr>  <fct>    <dbl>  <dbl>   <dbl>          <dbl>
 1 gengar             ghost  poison 1           60     65      60            130
 2 mismagius          ghost  <NA>   1           60     60      60            105
 3 tornadus-incarnate flying <NA>   1           79    115      70            125
 4 espeon             psych~ <NA>   1           65     65      60            130
 5 sceptile           grass  <NA>   1           70     85      65            105
 6 electabuzz         elect~ <NA>   1           65     83      57             95
 7 celebi             psych~ grass  1          100    100     100            100
 8 genesect           bug    steel  1           71    120      95            120
 9 girafarig          normal psych~ 1           70     80      65             90
10 froslass           ice    ghost  1           70     80      70             80
# i 509 more rows
# i 2 more variables: special_defense <dbl>, speed <dbl>
```

```
augmented_pokemon |>
  dplyr::count(.cluster, type_1) |>
  arrange(.cluster, desc(n))
```

```
# A tibble: 66 x 3
   .cluster type_1       n
   <fct>    <chr>    <int>
 1 1        fire        14
 2 1        electric    13
 3 1        psychic     13
 4 1        water       13
```

```
 5 1         grass       12
 6 1         bug         11
 7 1         normal      11
 8 1         dark         6
 9 1         dragon       6
10 1         ice          5
# i 56 more rows
```

```
augmented_pokemon |>
  dplyr::count(.cluster, type_2) |>
  arrange(.cluster, desc(n))
```

```
# A tibble: 63 x 3
   .cluster type_2       n
   <fct>    <chr>    <int>
 1 1        <NA>        60
 2 1        flying      23
 3 1        poison       7
 4 1        psychic      5
 5 1        fire         4
 6 1        dark         3
 7 1        dragon       3
 8 1        fighting     3
 9 1        ice          3
10 1        fairy        2
# i 53 more rows
```

```
augmented_pokemon |>
  pivot_longer(cols = c(type_1, type_2),
               names_to = "slot",
               values_to = "type") |>
  filter(!is.na(type)) |>
  dplyr::count(.cluster, type) |>
  group_by(.cluster) |>
  mutate(prop = n / sum(n)) |>
  arrange(.cluster, desc(prop))
```

```
# A tibble: 71 x 4
# Groups:   .cluster [4]
   .cluster type         n   prop
   <fct>    <chr>    <int>  <dbl>
```

```
 1 1        flying      24 0.136
 2 1        fire        18 0.102
 3 1        psychic     18 0.102
 4 1        electric    14 0.0795
 5 1        grass       13 0.0739
 6 1        water       13 0.0739
 7 1        bug         11 0.0625
 8 1        normal      11 0.0625
 9 1        poison      11 0.0625
10 1        dark         9 0.0511
# i 61 more rows
```

```
augmented_pokemon |>
  group_by(.cluster) |>
  summarise(across(hp:speed, mean))
```

```
# A tibble: 4 x 7
  .cluster    hp attack defense special_attack special_defense speed
  <fct>    <dbl>  <dbl>   <dbl>          <dbl>           <dbl> <dbl>
1 1         72.5   85.8    68.9           97.0            78.7  98.2
2 2         93.4   69.1    91.0           85.3           103.   53.3
3 3         85.1  108.     93.0           63.5            70.4  68.5
4 4         50.5   53.6    53.0           48.8            50.6  50.8
```

I chose 4 clusters just off intuition honestly. Now, cluster 1 appears to be fast special attackers, cluster 2 is bulky special tanks, cluster 3 are more physical attackers, and cluster 4 appears to be either just bad, statistically weak, and/or early/early-stage Pokemon. The fourth page created by this chunk keeps tally of how many secondary types in each cluster. This sort of supports the idea that that cluster could be filled with statistically weak, early-stage Pokemon as 120/217 of the Pokemon in that chunk do not have a secondary type. It is worth noting that most of the time when Pokemon have a Secondary-Type, it is developed in evolution. It is also worth noting that the total of Non-Secondary-Type Pokemon is 260, so nearly half of of the Non-Secondary-Type Pokemon are in cluster 4.

Part b

Explain the general concepts/procedure behind the algorithm you are implementing. Attempt as best you can to write your explanation in language that someone without much statistical knowledge can understand.

I used kmeans clustering. First, I grabbed the training set, selected the necessary variables, and labeled it as pokemon_clust. I then turned that into a matrix and scaled it. After setting

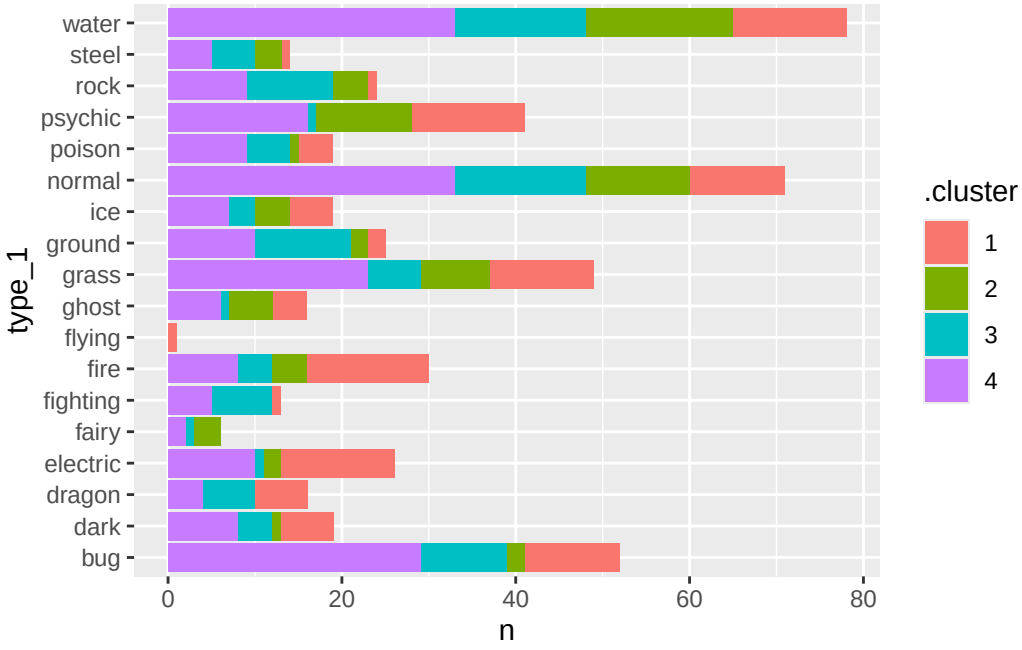
the seed, I use the kmeans function on the scaled matrix and set for 4 centers, nstart = 50 makes it so that we check 50 different centers and select the best. This clusters Pokemon that are statistically similar which is great. We then augment what the kmeans function produced with the pokemon_train dataset for a final needed dataset.

With that data set we check various things. We create a table that shows how much Pokemon are in each cluster, we also check the centers to see the average of each cluster to see what kind of Pokemon is found in each cluster. We create 2 tibbles that keep tally of how many of each Primary and Secondary Types in each cluster. Finally, the last tibble keeps track of the proportion of each type in each cluster.

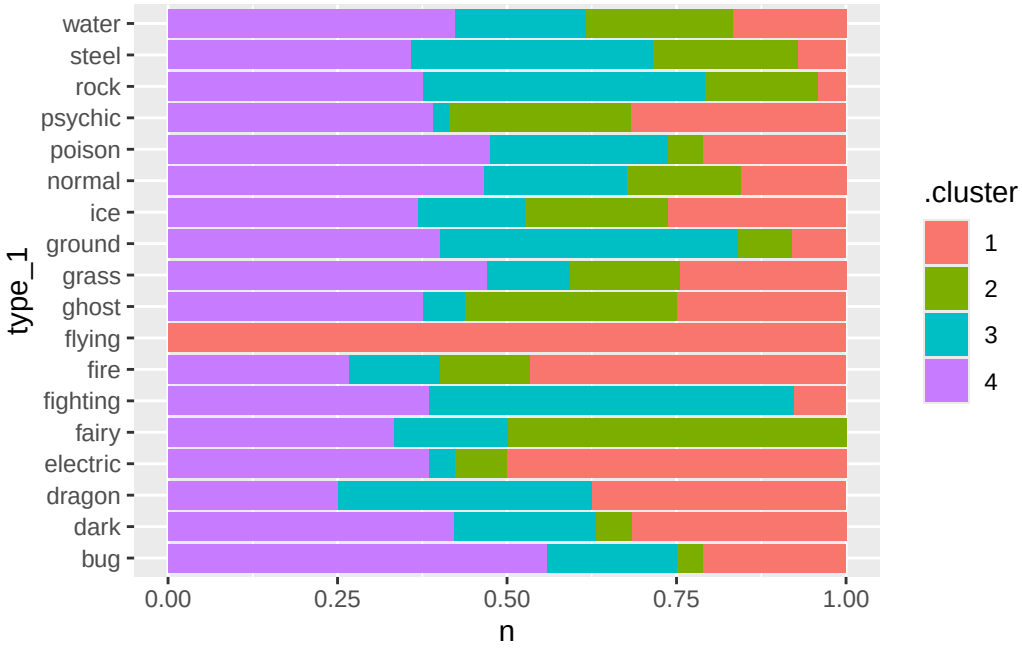
Part c

Produce appropriate graphs showing the results of your technique. Explain what your chosen PCs (at least the first two) or chosen clusters (at least two) represent in context.

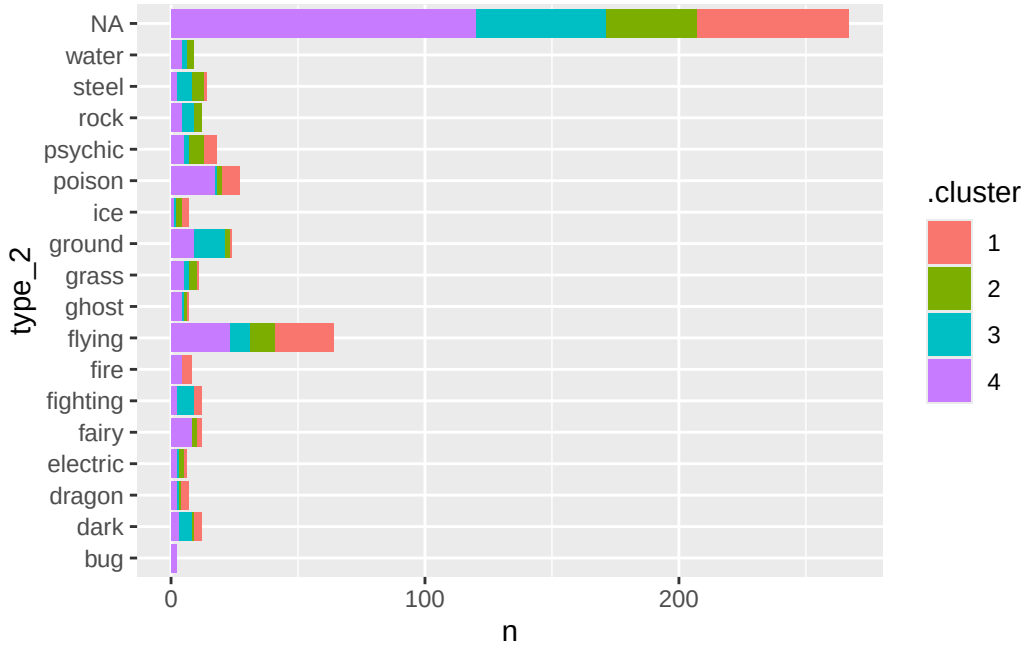
```
augmented_pokemon |>
  dplyr::count(.cluster, type_1) |>
  ggplot(aes(x = type_1, y = n, fill = .cluster)) +
  geom_col() +
  coord_flip()
```



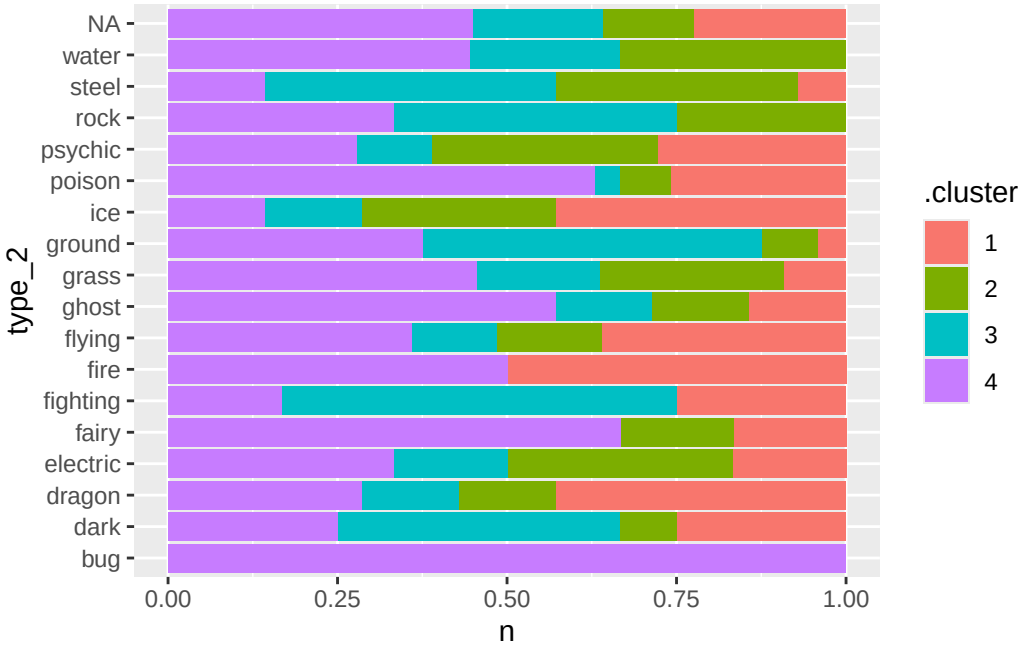
```
augmented_pokemon |>
  dplyr::count(.cluster, type_1) |>
  ggplot(aes(x = type_1, y = n, fill = .cluster)) +
  geom_col(position = "fill") +
  coord_flip()
```



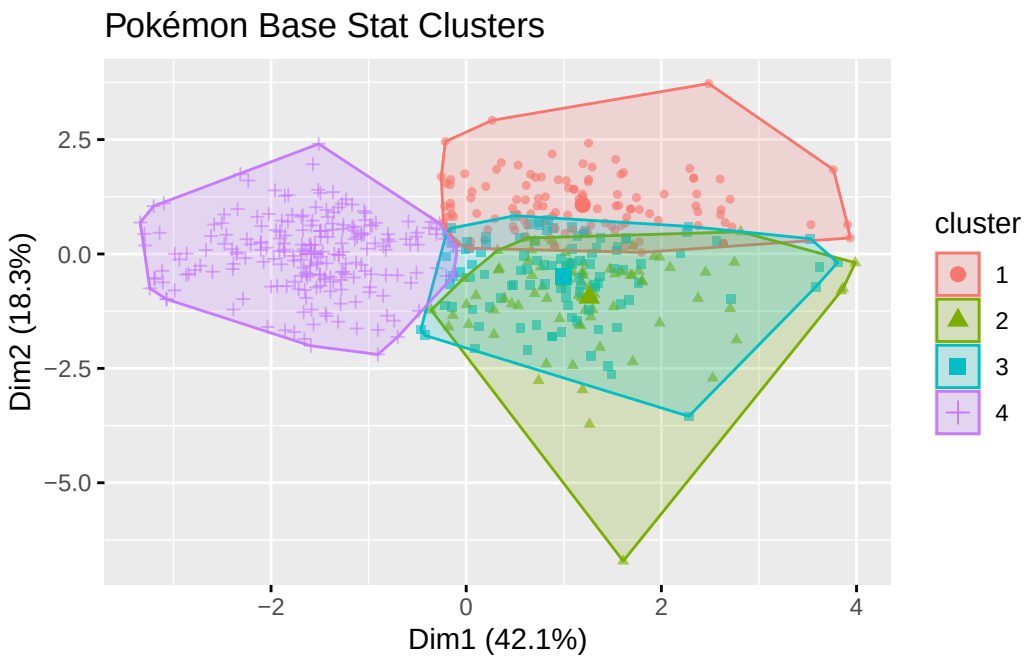

```
augmented_pokemon |>
  dplyr::count(.cluster, type_2) |>
  ggplot(aes(x = type_2, y = n, fill = .cluster)) +
  geom_col() +
  coord_flip()
```




```
augmented_pokemon |>
  dplyr::count(.cluster, type_2) |>
  ggplot(aes(x = type_2, y = n, fill = .cluster)) +
  geom_col(position = "fill") +
  coord_flip()
```




```
fviz_cluster(km.pokemon,
             data = x.matrix,
             geom = "point",      # This turns off the text/numbers
             pointsize = 1.25,      # Adjusts dot size
             alpha = 0.6) +
  labs(title = "Pokémon Base Stat Clusters")
```



Here we have various graphs. First 4 show how many of each type we see in each cluster both numerically and proportionately for both Primary and Secondary Types. The 5th graph shows us how certain Pokemon are distributed amongst the clusters, label by pokedex id, however, due to amount of observations, it is difficult to view displayed id numbers when we display them, so I just kept the points. As for interpreting the dimensions, it was difficult, had to get help, but, we can interpret the axes by comparing the positions of clusters in the plot. Since clusters with higher attack and speed appear to mostly be on the right, based on cluster 4, we could assume Dim1 reflects offensive strength, in terms of hp, attack, and defense maybe for example. With using cluster 4 as a basis of sorts again, I can hopefully assume Dim2 separates differences in stat distribution like special vs physical builds. Another reason for this safe assumption is special builds are generally low, so is is okay if chunk 4 sits with the majority, horizontal wise.

Section 4: Supervised Learning

Fit two supervised learning models on the training according to the following criteria: 1. At least one of the two models must be a new model introduced since Project #1 (i.e., k-nearest neighbors, bagging/random forests, or boosted trees for regression; k-nearest neighbors, LDA, QDA, Naive Bayes, or a tree-based method for classification). The other can be a linear/logistic regression model or a different "new model." 2. You should demonstrate your understanding of hyperparameter tuning by including and tuning at least one hyperparameter in at least one of the two models. If your dataset does not have a natural response variable, choose or create something that might make sense as a response variable.

Multinomial Linerar Regression Model K Nearest Neighbors Model (Tune Here)

Part a

Fit the models on the training set

```
set.seed(67)

# Convert response to factor
pokemon_train$type_1 <- as.factor(pokemon_train$type_1)

multi_model <- multinom_reg(
  mode = "classification",
  engine = "nnet"
)

multi_recipe <- recipe(
  type_1 ~ hp + attack + defense + special_attack + special_defense + speed,
  data = pokemon_train
)


multi_wflow <- workflow() |>
  add_model(multi_model) |>
  add_recipe(multi_recipe)

multi_fit <- fit(multi_wflow, data = pokemon_train)

multi_fit
```

```
== Workflow [trained] ==========================================================
Preprocessor: Recipe
Model: multinom_reg()

-- Preprocessor ----------------------------------------------------------------
0 Recipe Steps

-- Model ---------------------------------------------------------------------
Call:
nnet::multinom(formula = ..y ~ ., data = data, trace = FALSE)
```

```
Coefficients:
         (Intercept)           hp        attack      defense special_attack
dark      -1.0848929 -0.009934398  0.0292567655 -0.032024970    0.020307153
dragon    -4.2937918  0.003719990  0.0440860203 -0.026163355    0.027491407
electric  -1.4145423 -0.003031547 -0.0086419143 -0.041755116    0.049343477
fairy     -1.5207751  0.031542804 -0.0329485326 -0.022367451    0.048995535
fighting  -0.7337090  0.044287661  0.0439935004 -0.039702884   -0.038939770
fire      -2.0566650  0.014642723  0.0228317670 -0.045147137    0.046248512
flying    -3.2523877 -0.001121738  0.0188964867 -0.046096637    0.052512310
ghost     -1.7249900 -0.027804200 -0.0220178256 -0.004908067    0.067240387
grass      0.9594435  0.010347736 -0.0098610437 -0.022316488    0.049113314
ground    -2.0852761  0.040294896  0.0211540723  0.004942715   -0.021339422
ice       -1.4414800  0.032878983 -0.0015704348 -0.059387344    0.034835116
normal     0.2796489  0.055830371  0.0029186855 -0.051258069   -0.015097912
poison    -1.4922010  0.035902896 -0.0002854029 -0.004995321    0.001077835
psychic   -1.6288593  0.032964819 -0.0465187547 -0.033350953    0.068961012
rock      -2.2321131  0.002259195  0.0230146848  0.021007534    0.009756278
steel     -2.6789985  0.014843454 -0.0007472992  0.028313561    0.010091283
water     -0.3657149  0.043456668 -0.0212987914 -0.006841032    0.033153224
          special_defense        speed
dark       -0.0124600548  0.003243822
dragon     -0.0025799361 -0.007907373
electric   -0.0071221425  0.018451953
fairy       0.0103414122 -0.048547137
fighting   -0.0082733295 -0.030383333
fire       -0.0012233976 -0.016877533
flying     -0.0132385788 -0.009160374
ghost       0.0117317184 -0.023588135
grass      -0.0095196216 -0.029777622
ground     -0.0275841989 -0.006010240
ice         0.0205391755 -0.019471972
normal     -0.0009312282  0.007278079
poison     -0.0236386625  0.002287045
psychic     0.0019815252 -0.008200270
rock       -0.0190060048 -0.026290653
steel      -0.0170088935 -0.023481340
water      -0.0227638533 -0.009734301

Residual Deviance: 2288.182
AIC: 2526.182
```

```
knn_model <- nearest_neighbor(
  mode = "classification",
  neighbors = tune(),
  dist_power = 2
)

knn_recipe <- recipe(
  type_1 ~ hp + attack + defense + special_attack + special_defense + speed,
  data = pokemon_train
) |>
  step_normalize(all_numeric_predictors())

knn_wflow <- workflow() |>
  add_model(knn_model) |>
  add_recipe(knn_recipe)
```

Part b

Tune the hyperparameter(s) for one model

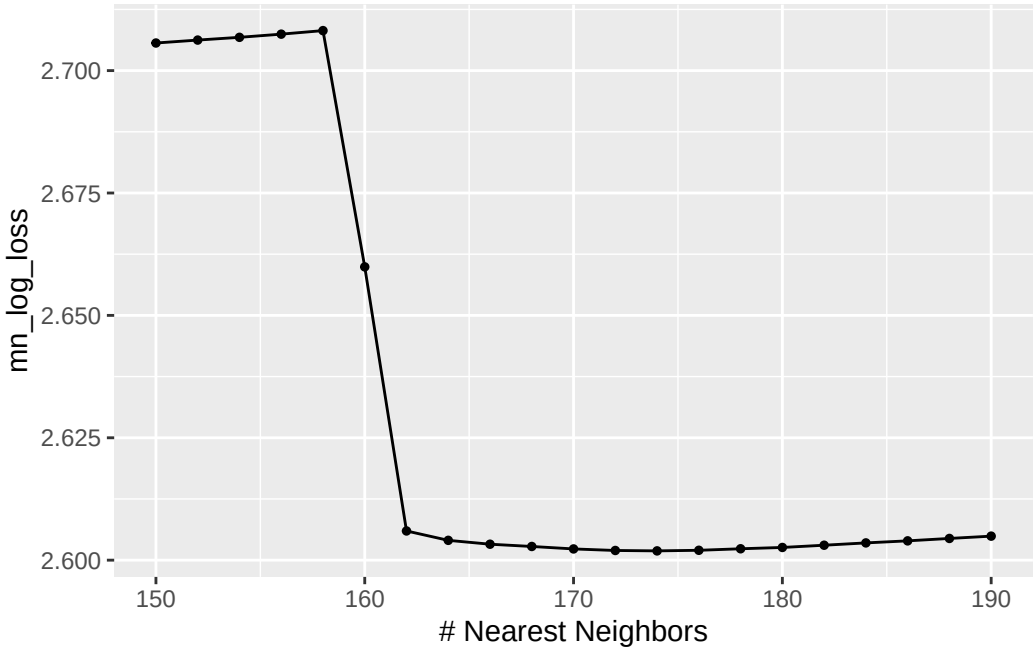
```
set.seed(67)

knn_kfold <- vfold_cv(pokemon_train, v = 5)

knn_grid <- expand.grid(
  neighbors = seq(150, 190, by = 2)     # Keep testing until it looks good
)

# Performs 5 Fold and computes metric, averages the results across folds
knn_tune <- tune_grid(
  knn_wflow,     # workflow to tune
  resamples = knn_kfold,     # tibble with instructions to k-fold cv
  grid = knn_grid,     # grid of hyperparameters values to search over
  metrics = metric_set(mn_log_loss)
)

autoplot(knn_tune)
```




```
best_k <- select_best(knn_tune, metric = "mn_log_loss")
best_k
```

```
# A tibble: 1 x 2
  neighbors .config
      <dbl> <chr>
1       174 pre0_mod13_post0
```

```
knn_model_final <- nearest_neighbor(
  mode = "classification",
  neighbors = best_k$neighbors,
  dist_power = 2
)

knn_wflow_final <- workflow() |>
  add_model(knn_model_final) |>
  add_recipe(knn_recipe)

knn_fit <- fit(knn_wflow_final,
               data = pokemon_train)
```

We chose the range of 150-190 after testing as the best k was in that ball park.

Part c

Explain what the hyperparameter(s) represent(s). Explain how the choice of one of the hyperparameters adjusts the bias vs. variance of the mode

The hyperparameter in k-nearest neighbors is the number of neighbors, k, which determines how many nearby observations are used when classifying a new Pokemon. When k is small, the model is very sensitive which leads to low bias but high variance. When k is large, we get high bias low variance. It is like a see-saw. Cross-validation is used to find a value of k that helps balance this tradeoff, which was determined by minimizing log-loss in my code.

Part d

Explain in general how each algorithm works. Attempt as best you can to write your explanation in language that someone without much statistical knowledge can understand. Identify the assumptions of the algorithm and assess (based on your EDA) whether the assumptions are reasonable.

It is worth noting that imbalance amongst types exists as some types are underrepresented, such as flying (we are doing just Primary-Types), which has one observation, so we will notice a heavy "skew" of sorts.

Multinomial logistic regression models the probability of each Pokemon type based off of each individual base stat. It predicts the probability that a Pokemon belongs to each possible type, but will only return the class for with the highest probability. The model works by comparing each class to a reference level or class, and then fitting linear relationships in the log-odds scale. one assumption of this model is that the relationship between predictors and log-odds is linear. We also assume independence between observations, which is vary reasonable in this case considering our observations are Pokemon.

K-nearest neighbors is a method that assigns a class based on the majority amongst the closest observations to the observation we are testing. Distance between Pokemon is computed using Euclidean distance on standardized stats. What this means is the distance between Pokemon is measured in a straight line from point a to point b. Standardized stats is like, the stats are adjusted to follow a normal distribution, so they are proportionally similar in a way. In short, KNN is assuming that Pokemon that are similar statistically would tend to share similar types. We have seen that this is somewhat reasonable in the Exploratory Data Analysis section. We saw that certain Pokemon types dominated or stayed in certain ranges with some offensive and defensive stats. We also saw some of this in the cluster graph at the end of section 3, but we did see overlap between clusters which might limits classification accuracy.

Part e

Using cross-validation, assess the accuracy and the standard deviation of average accuracy for each model.

Yeah could not figure this part in time, but considering what I mentioned in part d, accuracy is not going to be too hot.

In conclusion, maybe it is not possible to accurately predict a Pokemon's type based off their stats. But as Mewtwo said in his debut film, "I see now that the circumstances of one's birth are irrelevant; it is what you do with the gift of life that determines who you are."